the `CityBreaksContext` as a dependency. The initial code for the class is shown in the following listing.

Listing 8.14 The `CityService` class

```
using CityBreaks.Data;
using CityBreaks.Models;
using Microsoft.EntityFrameworkCore;

namespace CityBreaks.Services;
public class CityService : ICityService
{
    private readonly CityBreaksContext _context;
    public CityService(CityBreaksContext context) => _context = context;
    public async Task<List<City>> GetAllAsync()
    {
        var cities = _context.Cities                ❶
            .Include(c => c.Country)                ❷
            .Include(c => c.Properties);            ❷
        return await cities.ToListAsync();          ❸
    }
}
```

❶ The entry point to a query is the DbSet.

❷ Use the Include method to specify related data you want included in the query.

❸ The ToListAsync method call causes query execution.

LINQ queries consist of two phases: the *specification* phase and the *execution* phase. In our example, the specification of the query takes place in the first three lines of the `GetAllAsync` method. The execution is deferred until the last line, where you call the `ToListAsync` method. It is only at that point that a call to the database is made. This ability to defer execution enables you to continue to compose the specification by adding additional criteria. For example, you might want to filter the query to only return cities in France, which you can do within a `Where` method call that takes a predicate representing the filter condition(s):

```
var cities = _context.Cities
    .Where(c => c.Country.CountryName == "France")
```